

Task 6: Database Transactions & Concurrency Control

Scenario: The "Race Condition" in a 1-of-1 Vintage Marketplace.

Because RetroThreads deals exclusively in unique, single-inventory vintage garments, it is critical to maintain data consistency if two buyers attempt to purchase the same item simultaneously. To enforce ACID properties (specifically Isolation and Consistency), the application employs explicit database transactions combined with **Row-Level Locking** (`SELECT ... FOR UPDATE`).

Standard Transaction Flow:

When a buyer initiates a purchase, the system executes the following transaction block:

1. **Initiate Transaction:** `START TRANSACTION;`
2. **Acquire Exclusive Lock:** The system locks the specific listing row using `SELECT status FROM listing WHERE listing_id = X FOR UPDATE;`. This prevents any concurrent transactions from reading or modifying this row until the current transaction completes.
3. **Verify State:** The application confirms the `status` is still 'active'. If it is not, a `ROLLBACK` is issued.
4. **Execute Purchase:** `INSERT INTO transactions (listing_id, buyer_id, payment_status) VALUES (X, Y, 'completed');`
5. **Trigger Execution:** Several database triggers fire automatically during this phase:
 - `prevent_self_purchase`: Verifies the buyer is not the seller, throwing an error if true.
 - `set_transaction_price`: Ensures the `final_price` is accurately recorded by pulling it from the active listing.
 - `update_listing_status`: Updates the listing status to 'Sold' immediately after the transaction is recorded.
6. **Finalize:** If all steps and triggers succeed, the transaction is `COMMIT`ted, releasing the lock. If any step fails, the entire transaction is `ROLLBACK`ed, ensuring no partial data is written.

Basic code for showing in demo

Part 2: The Terminal Demonstration (The Conflicting Transaction)

Step 1: Buyer A starts the transaction and locks the dress. *(Run in Terminal 1)*

SQL

None

```
START TRANSACTION;  
SELECT status FROM listing WHERE listing_id = 5 FOR UPDATE;
```

Step 2: Buyer B tries to buy the exact same dress at the same time. *(Run in Terminal 2)*

SQL

None

```
START TRANSACTION;  
SELECT status FROM listing WHERE listing_id = 5 FOR UPDATE;
```

(Notice that Terminal 2 will completely freeze! The database is forcing Buyer B to wait because Buyer A has the lock).

Step 3: Buyer A finishes checking out. *(Run in Terminal 1)*

SQL

None

```
INSERT INTO transactions (listing_id, buyer_id, payment_status)  
VALUES (5, 3, 'completed');  
COMMIT;
```

Step 4: Buyer B gets rejected. (*Look at Terminal 2*) Terminal 2 will instantly unfreeze and output **sold**. Because the status is no longer 'active', Buyer B's transaction must be cancelled:

SQL

None

```
ROLLBACK;
```